

1. Write a C program to sort an array of n elements using Bubble Sort Technique.

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Bubble Sort Logic */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements:
45 12 78 3 25
Sorted array:
3 12 25 45 78
```

2. Write a C program to sort an array of n elements using Selection Sort Technique.

```
#include <stdio.h>
```

```

int main() {
    int n, i, j, min, temp;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Selection Sort Logic */
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min]) {
                min = j;
            }
        }

        if (min != i) {
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 5
Enter 5 elements:
45 12 78 3 25
Sorted array:
3 12 25 45 78

```

3. Write a C program to sort an array of n elements using Quick Sort Technique.

```
#include <stdio.h>
```

```
/* Function to swap two elements */
```

```

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* Partition function */
int partition(int a[], int low, int high) {
    int pivot = a[high]; /* choosing last element as pivot */
    int i = low - 1;
    int j;

    for (j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

/* Quick Sort function */
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);

        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n, i;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    quickSort(a, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {

```

```

        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements:
45 12 78 3 25 10
Sorted array:
3 10 12 25 45 78

```

4. Write a C program to sort an array of n elements using Simple Insertion Sort Technique.

```
#include <stdio.h>
```

```

int main() {
    int n, i, j, key;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Insertion Sort Logic */
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

```
}
```

Output:

Enter number of elements: 5

Enter 5 elements:

45 12 78 3 25

Sorted array:

3 12 25 45 78

5. Write a C program to sort an array of n elements using Shell Sort Technique.

```
#include <stdio.h>
```

```
int main() {
    int n, i, j, temp;
    int a[100];

    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    int m, gaps[50];
    scanf("%d", &m);
    for (i = 0; i < m; i++) scanf("%d", &gaps[i]);

    for (int g = 0; g < m; g++) {
        int gap = gaps[g];
        if (gap >= n) continue;

        for (i = gap; i < n; i++) {
            if (a[i - gap] > a[i]) {
                temp = a[i];
                a[i] = a[i - gap];
                a[i - gap] = temp;
            }
        }
    }

    for (i = gap; i < n; i++) {
        temp = a[i];
        j = i;
        while (j >= gap && a[j - gap] > temp) {
            a[j] = a[j - gap];
            j -= gap;
        }
        a[j] = temp;
    }
}

for (i = 0; i < n; i++) printf("%d ", a[i]);
```

```
    return 0;
}
```

Output:

```
Enter number of elements: 8
Enter 8 elements:
23 12 1 8 34 54 2 3
Enter number of gaps in the gap-sequence: 3
Enter 3 gap values:
4 2 1
Sorted array:
1 2 3 8 12 23 34 54
```

6. Write a C program to sort an array of n elements using Merge Sort Technique.

```
#include <stdio.h>

/* Function to merge two subarrays */
void merge(int a[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[100];

    while (i <= mid && j <= high) {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

/* Merge Sort function */
void mergeSort(int a[], int low, int high) {
    int mid;
```

```

        if (low < high) {
            mid = (low + high) / 2;

            mergeSort(a, low, mid);
            mergeSort(a, mid + 1, high);
            merge(a, low, mid, high);
        }
    }

int main() {
    int n, i;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    mergeSort(a, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements:
45 12 78 3 25 10
Sorted array:
3 10 12 25 45 78

```